Vodafone Tourist Pack

Frontend Documentation

Next.js tourist eSIM activation, custom plan builder, on-device passport scanning, PayPal checkout, and the Daily Drop reward game.

Prepared for internship demo

September 2026

1. Project Overview

The Vodafone Tourist Pack frontend is a Next.js 15 (App Router) application that lets a tourist visiting Albania find or build a mobile data pack, activate it with identity verification, pay through PayPal, receive an eSIM or physical SIM, add a Vodafone Tourist Pass to Apple/Google Wallet, and play a daily reward game.

It communicates with a separate Spring Boot backend over plain REST (no shared code, no monorepo tooling) and is organized by feature rather than by file type.

2. Tech Stack

- Next.js 15, App Router, Turbopack dev server
- TypeScript throughout
- Plain CSS (app/styles.css) with theme tokens via CSS custom properties and a [data-theme] attribute - no Tailwind, no CSS modules
- Framer Motion for the scratch-card and reveal animations
- Lucide for icons, plus a small set of hand-drawn purpose-built icons (signal bars, NFC wave) to avoid the generic-icon look
- PayPal JS SDK, proxied through Next.js API routes so the client secret never reaches the browser
- tesseract.js + mrz for 100% client-side passport/ID OCR

3. Architecture: Feature-First Structure

```
app/                  Routes: /, /activate, /payment, /game-hub, /my-pack,
                       /terms, /privacy, /wallet/{apple,google}/[id],
                       plus the PayPal API routes
features/
  activation/         Pack selection, custom plan builder, checkout stepper,
                      passport/ID scanner, PayPal, tourist details form
  game-hub/           Daily drop game, scratch card, reward reveal
  my-pack/            Subscription-status lookup + wallet button
  marketing/          Landing page offer sections
  stores/             Store locator map pins + data
shared/                Header, footer, theme toggle, shared icons/utils
```

Each feature owns its own lib/api.ts and talks to the backend directly - there is no shared, generated API client. TypeScript interfaces in features/activation/types/tourist.ts are maintained by hand to match the backend's DTOs.

4. Core User Flow

- / - browse 3 fixed packs, take a 4-question quiz for a recommendation, or build a custom plan
- /activate - enter tourist details (name, email, passport/ID - optionally auto-filled by the on-device scanner), pick eSIM or physical SIM delivery
- /payment - PayPal checkout; on successful capture, the backend activates the pack, provisions a (demo) eSIM, enrolls a PassKit Wallet member, and emails a confirmation
- /game-hub?touristId=... - claim a daily credit, scratch a card, win a real discount on the next pack purchase
- /my-pack?touristId=... - pack details, key dates, order info, and the Add to Apple Wallet button

touristId travels as a plain URL query parameter rather than a session cookie - there is no login flow by design, to keep activation friction-free for a short-stay tourist. It is an unguessable v4 UUID (not a sequential id), so it cannot practically be brute-forced, but a leaked link grants full access with no re-authentication check. That is an accepted trade-off for this app's low stakes (a discount code, SIM usage data - not payment instruments), not an oversight.

5. Feature: Custom Plan Builder

For tourists whose needs do not map cleanly onto one of the 3 fixed packs, a live-priced builder lets them choose data (metered GB or Unlimited), minutes, and validity via sliders. Price updates on every change via a debounced call to the backend's deterministic pricing endpoint (no client-trusted price, no AI-generated price - a plain, reproducible formula).

"Build & Continue" persists the choice as a real, hidden Pack row on the backend (is_active=false) and redirects into the exact same /activate?packId=... checkout flow already used for the 3 fixed packs - zero changes needed anywhere downstream (PayPal, activation, email, wallet all already read a Pack generically by id).

6. Feature: Passport/ID Scanning (privacy-first)

An optional "Scan Passport / ID" button autofills first name, last name, and document number - never email. The photo never leaves the browser:

- OCR (tesseract.js) and MRZ parsing (the mrz package) run entirely on-device.
- Only the passport's Machine Readable Zone is read - the standardized strip every airport e-gate reads - not the printed photo page.
- Only 3 fields are extracted; everything else the MRZ encodes (date of birth, sex, nationality, expiry date) is read but immediately discarded - data minimization by design.
- The photo, the canvas, and the full OCR text are all dropped the instant scanning finishes - nothing is written to any storage, client or server.
- A live camera view with an on-screen alignment guide is used when available (requires a secure context - HTTPS or localhost); it automatically falls back to a native camera/file picker with a static framing guide otherwise.

The resulting document number is still stored server-side exactly as if it had been typed manually - Albanian law requires a recorded identity document number for every SIM activation. Only the scanning mechanism is privacy-preserving; the legal record it produces is unchanged.

7. Feature: PayPal Payment - Two-Layer Architecture

PayPal calls never go directly from the browser to the backend. An intermediate Next.js API layer keeps secrets off the client and re-derives the price server-side on every order:

```
Browser
  -> POST /api/paypal/create-order   (Next.js edge route)
        - fetches the real pack price from the backend
        - checks for a returning-tourist discount
        - creates the PayPal order server-side
  -> PayPal modal (user approves)
  -> POST /api/paypal/capture-order  (Next.js edge route)
        - captures payment via PayPal's API
        - calls backend POST /api/v1/activations, only after capture succeeds
```

All three PayPal edge routes are rate-limited per client IP - tightest on capture-order, since it is the one that actually moves money.

8. Feature: Vodafone Tourist Pass (Apple/Google Wallet)

After activation, the backend enrolls the tourist as a PassKit member and returns a real wallet pass URL, surfaced as an "Add to Apple Wallet" button on /my-pack and linked from the confirmation email. If enrollment did not produce a pass (PassKit unreachable, misconfigured, or its free-tier issuance limit reached), the link falls back to a plain "wallet unavailable" notice page instead of a dead link.

The pass's QR code is generated automatically by PassKit from the member id - the app never renders its own barcode for this. An earlier version of the reward-claim screen did render a fake, non-scannable barcode; it was removed in favor of a link back to the real Tourist Pass, once it was confirmed the real pass already carries a genuine QR code.

9. Feature: Daily Drop Game

Every successful activation grants one game credit, redeemable once per day for a scratch card. The prize draw happens server-side the moment the tourist starts revealing the card (not after they finish scratching), so the real prize can be drawn on the canvas beneath the foil layer as it is scratched away, instead of a blank placeholder.

Winning a discount prize applies it automatically to the tourist's next pack purchase via the PayPal create-order flow's discount lookup.

10. Terms & Conditions / Privacy Policy

Two static, content-only pages (/terms, /privacy) linked from the checkout consent checkbox and the site footer. The privacy policy in particular documents, accurately, exactly how the passport scanner behaves - what is read, what is discarded, and what is retained and why.

11. Environment Variables

NEXT_PUBLIC_API_BASE_URL	Backend base URL, used by every feature's lib/api.ts. Defaults to http://localhost:8080/api/v1.
NEXT_PUBLIC_PAYPAL_CLIENT_ID	Public PayPal client id, safe to expose to the browser.
PAYPAL_CLIENT_SECRET	Server-side only - used by the PayPal edge routes, never sent to the client.
PAYPAL_MODE	sandbox or live; defaults to sandbox.

12. Known Limitations

- next.config.ts's allowedDevOrigins needs manual updating whenever the dev machine's LAN IP changes, or asset requests from a phone on the same Wi-Fi get silently blocked.
- PassKit's free/draft tier caps total passes issued and expires each one after 48 hours.
- The live camera passport-capture view requires a secure context (HTTPS/localhost) and is unavailable over a plain LAN IP - it degrades gracefully to a file-picker flow.
- Game Hub state is not polled or revalidated on window focus.
- The custom-plan pricing constants are placeholders pending real business sign-off.